МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БРЕСТКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ
Кафедра ИИТ

Отчет по лабораторной работе №4

Специальность ПО-12

Выполнил:
Студент 3 курса
Группы ПО-12
Мельник Р.С.
Проверила:
Кулик А. Д..

Брест 2026

Цель работы: научиться работать с Github API, приобрести практические навыки написания программ для работы с REST API или GraphQL API

Задание:
используя Github API, реализовать предложенное задание
на языке Python. Выполнить визуализацию результатов, с использованием графика или отчета. Можно использовать как REST API (рекомендуется), так и GraphQL

Вариант 3:Автоматическое отслеживание новых релизов и обновлений в репозитории GitHub

Условие:

Напишите Python-скрипт, который:

1. Запрашивает у пользователя список репозиториев GitHub для мониторинга (например, django/django, fastapi/fastapi, pallets/flask).

2. Использует GitHub API для получения информации о последних релизах и тегах в каждом репозитории.

3. Проверяет, появились ли новые версии или обновления с момента последнего запуска скрипта.

4. Если есть обновления, собирает следующую информацию:

 Номер новой версии

 Дата релиза

 Список изменений (changelog)

 Ссылку на страницу релиза

Код программы:

```
# pylint: disable=invalid-name
"""
Скрипт для автоматического отслеживания новых релизов в репозиториях GitHub.
Использует GitHub REST API и сохраняет состояние в JSON файл.
"""

import json
import os
import requests

STATE_FILE = "last_releases.json"



def get_latest_release(repo: str) -> dict:
    """
    Получает информацию о последнем релизе репозитория через GitHub API.
    """
    url = f"https://api.github.com/repos/{repo}/releases/latest"
    headers = {"Accept": "application/vnd.github.v3+json"}

    try:
        response = requests.get(url, headers=headers, timeout=10)
```

```
        if response.status_code == 200:
            return response.json()
        return None
    except requests.exceptions.RequestException as err:
        print(f"Ошибка сети для {repo}: {err}")
        return None


def load_state() -> dict:
    """Загружает данные о прошлых проверках из JSON файла."""
    if os.path.exists(STATE_FILE):
        with open(STATE_FILE, "r", encoding="utf-8") as f:
            return json.load(f)
    return {}


def save_state(state: dict) -> None:
    """Сохраняет текущие данные о версиях в JSON файл."""
    with open(STATE_FILE, "w", encoding="utf-8") as f:
        json.dump(state, f, ensure_ascii=False, indent=4)


def main() -> None:
    """Основная логика мониторинга обновлений."""
    user_input = input("Введите репозитории для отслеживания (через запятую): ")
    repo_list = [r.strip() for r in user_input.split(",") if r.strip()]
```

```
if not repo_list:
    print("Список репозиториев пуст.")
    return

last_state = load_state()
new_state = last_state.copy()

for repo in repo_list:
    print(f"\nПроверяем обновления для {repo}...")
    release_data = get_latest_release(repo)

    if not release_data:
        print(f"Информация о релизах в {repo} не найдена.")
        continue

    version = release_data.get("tag_name")
    last_seen_version = last_state.get(repo)

    if version != last_seen_version:
        date = release_data.get("published_at", "не указана")[:10]
        link = release_data.get("html_url")
        body = release_data.get("body", "Нет описания")
        changelog = "\n".join(body.splitlines()[:5])

        print(f"✅НАЙДЕН НОВЫЙ РЕЛИЗ: {version} ({date})")
```

```
            print(f"   Ссылка: {link}")
            print(f"   Основные изменения:\n{changelog}...")

            new_state[repo] = version
        else:
            print(f"□ Новых обновлений для {repo} нет (текущая: {version}).")

    save_state(new_state)
    print("\nПроверка завершена. Состояние сохранено в last_releases.json")


if __name__ == "__main__":
    main()
```

Результат:

```
c/task1.py
Введите репозитории для отслеживания (через запятую): Enixfai/Utils-for-c-

Проверяем обновления для Enixfai/Utils-for-c-...
✅ НАЙДЕН НОВЫЙ РЕЛИЗ: TEST (2026-03-30)
   Ссылка: https://github.com/Enixfai/Utils-for-c-/releases/tag/TEST
   Основные изменения:
TEST...

Проверка завершена. Состояние сохранено в last_releases.json
```

Вывод: научились работать с Github API, приобрели практические навыки написания программ для работы с REST API и GraphQL API